CS285 Homework 4

Model-Based Reinforcement Learning

2026 年 7 月 12 日

目录

1 开场：先用两分钟讲清楚整份作业 **4**

1.1 一张图记住 Problem 1 到 Problem 6 的关系 5

2 Analysis Problem 2.1：Alternative Simulation Lemma **5**

2.1 题目要求与代码对应关系 5

2.2 完整证明 6

3 Analysis Problem 2.2：Concentration Bound 判断 **7**

3.1 题目要求与代码对应关系 7

3.2 核心判断标准 7

3.3 逐项判断 8

4 Model-Based RL Problem 1：训练 Dynamics Model **8**

4.1 题目要求 8

4.2 代码一：更新第 i 个 dynamics model 9

4.3 代码二：更新 normalization 统计量 10

4.4 代码三：训练 ensemble 中的每一个模型 11

4.5 代码逐段怎么讲 11

4.6 实验结果 12

4.7 结果分析讲稿 13

5 Model-Based RL Problem 2：模型预测与 Random Shooting MPC **13**

5.1 题目要求 13

5.2 代码一：预测下一状态 13

5.3 代码二：评估所有候选动作序列 14

5.4 代码三：Random Shooting MPC 16

5.5 现场逐步讲解 17

5.6 实验结果与分析 17

6 Model-Based RL Problem 3：On-Policy 数据收集 **18**

6.1 题目要求 . . . 18
6.2 对应代码 . . . 18
6.3 代码逻辑讲解 . . . 19
6.4 实验结果 . . . 19
6.5 结果分析 . . . 20

7 Model-Based RL Problem 4：MPC 超参数消融 21
7.1 题目要求 . . . 21
7.2 对应 YAML 配置 . . . 21
7.3 完整结果 . . . 22
7.4 Ensemble size：代码参数如何影响算法 . . . 22
7.5 Candidate action sequences：随机搜索覆盖率 . . . 23
7.6 Planning horizon：看得更远与误差累积的权衡 . . . 24

8 Model-Based RL Problem 5：Cross-Entropy Method 25
8.1 题目要求 . . . 25
8.2 完整核心代码 . . . 26
8.3 逐轮算法过程 . . . 27
8.4 现场怎么讲代码 . . . 28
8.5 实验结果 . . . 28
8.6 结果分析 . . . 29

9 Model-Based RL Problem 6：Dyna-Style 与 MBPO 29
9.1 题目要求 . . . 29
9.2 代码一：使用 learned model 生成 synthetic rollout . . . 30
9.3 代码二：将 synthetic transitions 加入 SAC replay buffer . . . 31
9.4 数据流与形状 . . . 32
9.5 现场逐段讲解 . . . 32
9.6 实验结果 . . . 33
9.7 结果分析 . . . 34

10 把六道题连成一个完整故事 34
10.1 从代码角度看完整数据流 . . . 34

10.2 所有关键结果汇总 35

11 个人思考与总结 35

11.1 Model-Based RL 的真正优势 35

11.2 贯穿全部实验的风险：模型误差累积 36

11.3 为什么 uncertainty-aware 方法值得继续做 36

11.4 最终收尾. 36

A 可能的问题 37

1 开场：先用两分钟讲清楚整份作业

这次作业的主线是一条逐步搭建起来的完整链条：

第一，Problem 1 训练 dynamics model；

第二，Problem 2 用学到的模型预测未来，并实现 Random Shooting MPC；

第三，Problem 3 使用当前 MPC 策略继续收集 on-policy data，让模型不断适应新的状态分布；

第四，Problem 4 分析 ensemble size、候选动作序列数量和 planning horizon 三个超参数；

第五，Problem 5 用 CEM 代替一次性 random shooting，提高动作序列搜索效率；

第六，Problem 6 不再让模型直接控制，而是让模型生成 synthetic transitions，辅助 SAC 训练，也就是 MBPO。

1.1 一张图记住 Problem 1 到 Problem 6 的关系

随机策略采集真实数据

⇓

Problem 1：训练 ensemble dynamics model

⇓

Problem 2：模型 rollout + Random Shooting MPC

⇓

Problem 3：当前 MPC 策略收集 on-policy data，再训练模型

⇓

Problem 4：分析 ensemble、候选序列数和 horizon

⇓

Problem 5：用 CEM 改进动作序列搜索

⇓

Problem 6：模型生成 synthetic transitions，训练 SAC

整份作业的核心矛盾：**模型使用得越充分，越能减少真实环境试错；但模型预测得越远，误差累积越严重**。

2 Analysis Problem 2.1：Alternative Simulation Lemma

2.1 题目要求与代码对应关系

本题要求证明：对任意固定策略 π，真实 MDP 与经验模型 MDP 的动作价值函数满足

$$Q^{\pi} - \widehat{Q}^{\pi} = \gamma \left(I - \gamma P^{\pi}\right)^{-1} \left(P - \widehat{P}\right)\widehat{V}^{\pi}.$$

2.2 完整证明

真实 MDP 中，对固定策略 π，Bellman 方程为

$$Q^\pi = r + \gamma P V^\pi.$$

经验模型中，转移概率为 $\widehat{P}$，因此

$$\widehat{Q}^\pi = r + \gamma \widehat{P}\widehat{V}^\pi.$$

两式相减，reward 项抵消：

$$Q^\pi - \widehat{Q}^\pi = \gamma P V^\pi - \gamma \widehat{P}\widehat{V}^\pi.$$

为了将误差拆成“价值函数传播误差”和“单步模型误差”，在右侧加上再减去同一个量 $\gamma P\widehat{V}^\pi$：

$$\begin{aligned} Q^\pi - \widehat{Q}^\pi &= \gamma P V^\pi - \gamma P\widehat{V}^\pi + \gamma P\widehat{V}^\pi - \gamma \widehat{P}\widehat{V}^\pi \\ &= \gamma P(V^\pi - \widehat{V}^\pi) + \gamma(P - \widehat{P})\widehat{V}^\pi. \end{aligned}$$

由于 V^π 是 Q^π 在策略 π 下对动作取期望，所以

$$V^\pi - \widehat{V}^\pi = \Pi^\pi(Q^\pi - \widehat{Q}^\pi).$$

把策略选择动作的操作与环境转移结合，可以写成

$$P(V^\pi - \widehat{V}^\pi) = P^\pi(Q^\pi - \widehat{Q}^\pi).$$

代回前式：

$$Q^\pi - \widehat{Q}^\pi = \gamma P^\pi(Q^\pi - \widehat{Q}^\pi) + \gamma(P - \widehat{P})\widehat{V}^\pi.$$

移项：

$$(I - \gamma P^\pi)(Q^\pi - \widehat{Q}^\pi) = \gamma(P - \widehat{P})\widehat{V}^\pi.$$

因为 $\gamma \in (0, 1)$，且 P^π 是随机转移矩阵，所以 $I - \gamma P^\pi$ 可逆。两边左乘逆矩阵：

$$Q^\pi - \widehat{Q}^\pi = \gamma(I - \gamma P^\pi)^{-1}(P - \widehat{P})\widehat{V}^\pi.$$

证明完成。

2.3 分析

这一题的证明表面上是在做矩阵代数，但真正重要的是它揭示了模型误差如何影响价值函数。
右侧的 $(P-\widehat{P})\widehat{V}^{\pi}$ 表示单步模型误差：真实环境会按照 P 转移，而 learned model 按照 $\widehat{P}$ 转移，这两个结果不同。
前面的 $\gamma(I-\gamma P^{\pi})^{-1}$ 表示误差会沿着策略未来访问的轨迹反复传播。因为

$$(I-\gamma P^{\pi})^{-1}=I+\gamma P^{\pi}+\gamma^{2}(P^{\pi})^{2}+\cdots,$$

所以它实际上包含了第一步、第二步、第三步以及更远未来的误差传播。
因此，这个理论公式提前解释了后面的实验结果：horizon 越长，并不一定越好；model rollout 越长，也不一定一直带来收益。因为单步误差会被一层一层传下去。

“为什么逆矩阵存在?”因为 P^{π} 的谱半径不超过 1，而 $\gamma<1$，所以 γP^{π} 的谱半径小于 1，Neumann series 收敛，因此 $I-\gamma P^{\pi}$ 可逆。

3 Analysis Problem 2.2：Concentration Bound 判断

3.1 题目要求与代码对应关系

本题要求判断四个 concentration bound 哪些正确，并说明 Hoeffding inequality 的使用条件是否满足。

3.2 核心判断标准

题目中考虑的随机变量形式类似于

$$X_i=\mathbf{1}_{s_i'}\cdot V.$$

使用 Hoeffding inequality 时，需要把 V 看成固定向量。若 V 本身又由同一批 transition samples 计算得到，那么 X_i 与数据之间已经耦合，不能直接把它们当成独立有界随机变量。

3.3 逐项判断

选项 1 正确。 它先使用范数不等式：

$$\|(P-\widehat{P})\widehat{V}^{\pi}\|_{\infty} \le \max_{s,a} \|P(\cdot|s,a)-\widehat{P}(\cdot|s,a)\|_1 \cdot \|\widehat{V}^{\pi}\|_{\infty}.$$

由于 reward 在 $[0,1]$，有

$$\|\widehat{V}^{\pi}\|_{\infty} \le \frac{1}{1-\gamma}.$$

这里 concentration bound 真正控制的是 transition probability 的 L_1 误差。一旦这个上界成立，它可以作用于任意 value vector，因此不要求 $\widehat{V}^{\pi}$ 与样本独立。

选项 2 错误。 它试图直接对 $(P-\widehat{P})\widehat{V}^{\pi}$ 使用 Hoeffding。但是 $\widehat{V}^{\pi}$ 是从经验模型 $\widehat{P}$ 计算出来的，而 $\widehat{P}$ 又来自同一批样本。因此 $\widehat{V}^{\pi}$ 不是独立于样本的固定向量。

选项 3 正确。 这里使用的是真实 MDP 的最优价值函数 V^*。在估计 $\widehat{P}$ 的采样过程中，V^* 由真实环境 P 决定，可以看成固定向量，所以可以直接使用 Hoeffding。

选项 4 错误。 $\widehat{V}^*$ 是经验 MDP 的最优价值函数，依赖 $\widehat{P}$，因此仍然依赖当前采样数据，和选项 2 有相同问题。

> 这一题最容易犯的错误，是只看 value function 有没有上界，却没有检查它是不是由同一批数据计算出来的。
> 选项 3 中的 V^* 来自真实 MDP，所以在估计经验模型时可以把它看成固定对象；但选项 4 中的 $\widehat{V}^*$ 来自经验 MDP，本身就随着采样结果变化。
> 因此，Hoeffding inequality 不是“看到经验均值就可以直接套”，而是必须先检查随机变量之间的独立性和数据依赖关系。

4 Model-Based RL Problem 1：训练 Dynamics Model

4.1 题目要求

Problem 1 要求完成以下工作：

1. 使用随机策略采集真实 transition；
2. 在 `model_based_agent.py` 中实现 dynamics model 的更新；

3. 实现 normalization 统计量的更新；
4. 在 `run_hw4.py` 中训练 ensemble 中每一个 dynamics model；
5. 比较默认网络、加宽网络和加深网络三种配置的 dynamics loss。

模型不是直接预测 s_{t+1}，而是预测状态差分：

$$\Delta s_t = s_{t+1} - s_t.$$

模型形式为

$$\widehat{\Delta s_t} = f_\theta(s_t, a_t), \qquad \widehat{s}_{t+1} = s_t + \widehat{\Delta s_t}.$$

4.2 代码一：更新第 i 个 dynamics model

```
def update(self, i: int, obs, acs, next_obs):
    # 将 replay buffer 中的 numpy 数组转为 torch Tensor。
    obs = ptu.from_numpy(obs)
    acs = ptu.from_numpy(acs)
    next_obs = ptu.from_numpy(next_obs)

    # 取出 ensemble 中第 i 个 dynamics model 对应的 optimizer。
    optimizer = self.dynamics_optimizers[i]

    # 清空上一轮反向传播留下的梯度。
    optimizer.zero_grad()

    # 防止除以接近 0 的标准差。
    eps = 1e-8

    # 将当前状态和动作拼接成模型输入 [s_t, a_t]。
    obs_acs = torch.cat([obs, acs], dim=-1)

    # 使用 replay buffer 的统计量对输入做 normalization。
    obs_acs_normalized = (
        obs_acs - self.obs_acs_mean
    ) / (self.obs_acs_std + eps)

    # 模型监督目标是状态差分，而不是完整下一状态。
    obs_delta = next_obs - obs

    # 对状态差分做 normalization。
    obs_delta_normalized = (
        obs_delta - self.obs_delta_mean
    ) / (self.obs_delta_std + eps)

    # 第 i 个模型预测 normalized state delta。
    pred_obs_delta_normalized = self.dynamics_models[i](
        obs_acs_normalized
    )

    # 使用均方误差比较预测 delta 和真实 delta。
    loss = self.loss_fn(
```

```
        pred_obs_delta_normalized,
        obs_delta_normalized,
    )

    # 反向传播并更新模型参数。
    loss.backward()
    optimizer.step()

    # 返回 Python 标量，便于日志记录和绘图。
    return loss.item()
```

张量/数组形状检查

`obs`	(B, d_s)，其中 B 是 batch size，d_s 是状态维度；
`acs`	(B, d_a)；
`next_obs`	(B, d_s)；
`obs_acs`	$(B, d_s + d_a)$；
`obs_delta`	(B, d_s)；
`pred_obs_delta_normalized`	(B, d_s)。

4.3 代码二：更新 normalization 统计量

```
@torch.no_grad()
def update_statistics(self, obs, acs, next_obs):
    # numpy 转为 torch Tensor。
    obs = ptu.from_numpy(obs)
    acs = ptu.from_numpy(acs)
    next_obs = ptu.from_numpy(next_obs)

    # 拼接 observation 和 action，形成模型输入。
    obs_acs = torch.cat([obs, acs], dim=-1)

    # 计算模型监督目标 delta = next_obs - obs。
    obs_delta = next_obs - obs

    # 更新输入 [s_t, a_t] 的均值和标准差。
    self.obs_acs_mean.copy_(obs_acs.mean(dim=0))
    self.obs_acs_std.copy_(
        obs_acs.std(dim=0, unbiased=False)
    )

    # 更新状态差分的均值和标准差。
    self.obs_delta_mean.copy_(obs_delta.mean(dim=0))
    self.obs_delta_std.copy_(
        obs_delta.std(dim=0, unbiased=False)
    )
```

4.4 代码三：训练 ensemble 中的每一个模型

```
# 当前训练 step 中，每个 ensemble model 都会得到一个 loss。
step_losses = []

# 遍历 ensemble 中的所有 dynamics models。
for i in range(mb_agent.ensemble_size):

    # 为第 i 个模型单独采样一个 mini-batch。
    # 不同模型看到的数据不同，有助于形成模型多样性。
    batch = replay_buffer.sample(
        config["train_batch_size"]
    )

    # 更新第 i 个 dynamics model。
    loss = mb_agent.update(
        i,
        batch["observations"],
        batch["actions"],
        batch["next_observations"],
    )

    # 保存该模型的 loss。
    step_losses.append(loss)

# 用 ensemble 平均 loss 作为当前训练 step 的日志。
all_losses.append(np.mean(step_losses))
```

4.5 分 析

这部分首先要说明，网络不是直接预测 next observation，而是预测 observation delta。代码中的关键行是

$$\texttt{obs_delta = next_obs - obs}.$$

在连续控制环境中，相邻时刻状态往往变化不大，因此状态差分比完整下一状态更加平滑。例如当前位置是 5.0，下一时刻是 5.03，与其让网络从头预测 5.03，不如让它预测“增加了 0.03”。

第二个重点是 normalization。状态的不同维度可能具有完全不同的数量级，例如位置可能在 $[-1, 1]$，速度可能在几十的范围。如果不做标准化，数值较大的维度可能主导 loss 和梯度。代码分别保存了输入 $[s_t, a_t]$ 和目标 $s_{t+1} - s_t$ 的均值、标准差。

第三个重点是 ensemble。这里不是一个模型更新多次，而是有多个独立模型。每个模型有不同初始化，并且每次单独采样 mini-batch。这样它们不会完全学成同一个函数。规划时对多个模型预测取平均，可以降低单个模型偶然预测错误的影响。

常见错误 1： 把 `next_obs` 直接作为训练目标，却在预测函数中又执行 `obs + pred_delta`，会导致语义不一致。
常见错误 2： 输入做了 normalization，但 target 没做，或者训练时做了 target normalization，预测时忘记反归一化。
常见错误 3： 所有 ensemble models 每一步都使用完全相同的 batch，会削弱 ensemble 多样性。

4.6 实验结果

表 1: Problem 1 三组 dynamics model 配置与最终 loss

配置	隐藏层数	hidden size	最终 dynamics loss
Default	1	32	0.1623
Wider	1	64	0.1177
Deeper	2	32	0.1542



图 1: Problem 1：三组 dynamics model 的训练 loss 曲线。

4.7 结果分析

从三条曲线可以看到，loss 在训练初期快速下降，然后逐渐稳定，说明模型确实从随机动作数据中学到了基本的一步动力学规律。
默认配置是一层隐藏层、hidden size 为 32，最终 loss 为 0.1623，低于作业要求的 0.2，因此默认实现达到要求。
将 hidden size 从 32 增加到 64 后，最终 loss 降低到 0.1177，是三组中最低的。这说明在当前固定数据集和训练步数下，增加网络宽度提高了拟合能力。
把网络从一层增加到两层后，最终 loss 为 0.1542，也比默认值低，但改善没有加宽网络明显。这并不说明深层网络一定更差，而是说明更深模型可能需要不同学习率、更多训练步数或者更合适的正则化。当前实验中，增加宽度是更直接的改进。
还要强调：training loss 更低不一定保证控制效果一定更好。因为真正重要的是模型在 MPC 会访问的状态区域中是否准确，而不仅是它在训练 batch 上的平均 MSE。

5 Model-Based RL Problem 2：模型预测与 Random Shooting MPC

5.1 题目要求

Problem 2 要求完成三部分代码：

1. 用第 i 个 dynamics model 预测下一状态；
2. 对每条候选动作序列进行长度为 H 的模型 rollout，计算累计 reward；
3. 实现 Random Shooting，在 K 条候选序列中选择预测回报最高的一条，并只执行第一个动作。

5.2 代码一：预测下一状态

```
@torch.no_grad()
def get_dynamics_predictions(self, i: int, obs, acs):
    # numpy 转为 torch Tensor。
    obs = ptu.from_numpy(obs)
    acs = ptu.from_numpy(acs)

    eps = 1e-8

    # 拼接当前状态和动作。
    obs_acs = torch.cat([obs, acs], dim=-1)
```

```
        # 使用 Problem 1 保存的统计量做 normalization。
        obs_acs_normalized = (
            obs_acs - self.obs_acs_mean
        ) / (self.obs_acs_std + eps)

        # 第 i 个模型输出 normalized delta。
        pred_obs_delta_normalized = self.dynamics_models[i](
            obs_acs_normalized
        )

        # 将 normalized delta 反归一化到真实状态尺度。
        pred_obs_delta = (
            pred_obs_delta_normalized
            * (self.obs_delta_std + eps)
            + self.obs_delta_mean
        )

        # 当前状态加预测状态变化量，得到下一状态。
        pred_next_obs = obs + pred_obs_delta

        # MPC 使用 numpy，因此转回 numpy。
        return ptu.to_numpy(pred_next_obs)
```

训练阶段网络学习的是 normalized delta，所以预测阶段必须按相反顺序处理：

$$\text{normalized input} \to \text{normalized predicted delta} \to \text{unnormalized delta} \to s_t + \widehat{\Delta s_t}.$$

5.3 代码二：评估所有候选动作序列

```
def evaluate_action_sequences(self, obs, action_sequences):
    # 当前 observation 被复制到 ensemble 和 sequence 两个维度。
    # 形状变为 [ensemble_size, num_sequences, ob_dim]。
    obs = np.tile(
        obs[None],
        (
            self.ensemble_size,
            self.mpc_num_action_sequences,
            1,
        ),
    )

    # 保存每个模型对每条动作序列的累计 reward。
    pred_rewards = np.zeros(
        (
            self.ensemble_size,
            self.mpc_num_action_sequences,
        )
    )

    # action_sequences 原形状为 [K, H, ac_dim]。
    # 转置后按时间步遍历，每次 acs 的形状为 [K, ac_dim]。
```

```
    for acs in np.transpose(action_sequences, (1, 0, 2)):

        # 每个 dynamics model 都预测 K 条序列当前一步的 next_obs。
        next_obs = np.stack(
            [
                self.get_dynamics_predictions(
                    i,
                    obs[i],
                    acs,
                )
                for i in range(self.ensemble_size)
            ],
            axis=0,
        )

        # 动作复制到 ensemble 维度。
        acs_repeated = np.tile(
            acs[None, :, :],
            (self.ensemble_size, 1, 1),
        )

        # 用环境提供的 reward function 计算预测 transition 的 reward。
        rewards, _ = self.env.get_reward(
            next_obs.reshape(-1, self.ob_dim),
            acs_repeated.reshape(-1, self.ac_dim),
        )

        # 恢复成 [ensemble_size, num_sequences]。
        rewards = rewards.reshape(
            self.ensemble_size,
            self.mpc_num_action_sequences,
        )

        # 累加当前时间步 reward。
        pred_rewards += rewards

        # 下一步从刚刚预测出的 next_obs 继续 rollout。
        obs = next_obs

    # 对 ensemble 维度取平均，得到每条动作序列的最终评分。
    return pred_rewards.mean(axis=0)
```

张量/数组形状检查

设 ensemble size 为 E，候选序列数为 K，规划长度为 H。

`action_sequences`	(K, H, d_a)；
转置后每个 `acs`	(K, d_a)；
复制后的 `obs`	(E, K, d_s)；
`next_obs`	(E, K, d_s)；
`rewards`	(E, K)；
最终返回值	$(K,)$，每个元素是一条候选序列的 ensemble 平均累计 reward。

5.4 代码三：Random Shooting MPC

```
# 随机生成 K 条动作序列，每条长度为 H。
action_sequences = np.random.uniform(
    self.env.action_space.low,
    self.env.action_space.high,
    size=(
        self.mpc_num_action_sequences,
        self.mpc_horizon,
        self.ac_dim,
    ),
)

if self.mpc_strategy == "random":
    # 计算每条动作序列的 predicted cumulative reward。
    rewards = self.evaluate_action_sequences(
        obs,
        action_sequences,
    )

    # 找到预测累计 reward 最大的动作序列。
    best_action_sequence_idx = np.argmax(rewards)
    best_action_sequence = action_sequences[
        best_action_sequence_idx
    ]

    # MPC 只执行最优序列的第一个动作。
    return best_action_sequence[0]
```

5.5 分析

Random Shooting MPC 可以理解成“先在模型世界里试很多种未来，再从中选最好的一种”。
假设候选序列数 $K = 1000$，horizon $H = 10$。代码会一次生成 1000 条长度为 10 的动作序列。
对于每一条序列，模型从当前真实状态开始，连续预测 10 步状态，并累加
10 步 reward。
这里为什么要将 observation 复制成 $[E, K, d_s]$? 因为我们既有 E 个 ensemble models, 又有 K 条候选序列。每个模型都要独立预测每条序列的未来, 所以需要同时保留模型维度和序列维度。
最后，对 ensemble 维度取平均，得到每条动作序列的评分。选出评分最高的序列以后，代码只返回它的第一个动作，而不是一次执行完整的 10 步。
只执行第一步的原因是：执行动作后，我们能够获得真实环境的新状态。下一时刻重新规划，就可以用真实状态修正上一轮模型预测产生的偏差。这就是 receding horizon
control，也是 MPC 能够减弱长期模型误差的一种机制。

“既然只执行第一步，为什么还要预测后面 $H - 1$ 步?”因为当前动作的好坏不能只看即时 reward。一个动作可能眼前收益不高，但能把系统带到更有利的未来状态。预测多步是为了评估当前动作的长期影响；只执行一步是为了防止完全依赖不准确的长轨迹预测。

5.6 实验结果与分析

在 Obstacles 环境中，一轮训练后的 Random Shooting MPC 得到

$$\text{Eval Return} = -30.26.$$

作业要求结果大于 -70，因此达到要求。

这个结果说明，dynamics model 不需要完美重建每个未来状态，也可以对控制有帮助。MPC 更关心不同动作序列之间的相对排序。如果模型能够大致判断某条序列比另一条序列更好，即使绝对状态预测存在误差，也可能选出有效动作。
所以 Problem 2 是从“模型 loss 是否下降”迈向“模型是否真的能用于控制”的关键一步。
Problem 1 只证明模型能够拟合数据，Problem 2 才证明模型预测能够转化成实际 return。

6 Model-Based RL Problem 3：On-Policy 数据收集

6.1 题目要求

Problem 3 不增加新的核心 TODO，而是将 Problem 1 和 Problem 2 连接成完整迭代流程：

1. 第 0 轮用 random policy 收集初始数据；
2. 训练 dynamics ensemble；
3. 使用当前 ModelBasedAgent，也就是 MPC policy，收集新轨迹；
4. 将新数据加入 replay buffer；
5. 更新 normalization 统计量并继续训练模型；
6. 每轮进行 evaluation。

6.2 对应代码

```
# 第 0 轮还没有可靠的 dynamics model，因此先使用随机策略。
if itr == 0:
    trajs, envsteps_this_batch = utils.sample_trajectories(
        env,
        utils.RandomPolicy(env),
        config["initial_batch_size"],
        ep_len,
    )

# 从第 1 轮开始，使用当前的 ModelBasedAgent 收集数据。
else:
    trajs, envsteps_this_batch = utils.sample_trajectories(
        env,
        actor_agent,
        config["batch_size"],
        ep_len,
    )

# 将新采集的轨迹加入 replay buffer，旧数据不会被立即丢弃。
replay_buffer.insert_trajs(trajs)

# 使用不断增长的数据集重新计算 normalization 统计量。
mb_agent.update_statistics(
    replay_buffer.observations,
    replay_buffer.actions,
    replay_buffer.next_observations,
)

# 接下来继续执行 Problem 1 中的 ensemble training loop。
```

6.3 代码逻辑分析

Problem 3 的关键不是新增一个复杂函数，而是理解数据分布在训练过程中会发生变化。

第 0 轮使用 random policy，是因为此时模型还没训练，MPC 没有可信的预测能力。随机策略虽然控制效果不好，但能够提供最初的一批 transition，让模型先学到环境的大致变化规律。

从第 1 轮开始，数据由当前 actor agent 收集。在这里 actor agent 就是 ModelBasedAgent，也就是使用 MPC 选动作的智能体。随着模型变好，MPC 会访问越来越不同的状态区域。

每一轮新数据都会插入 replay buffer，然后使用整个 buffer 重新计算 normalization 统计量并继续训练模型。于是形成闭环：模型决定策略访问哪里，策略产生的数据又决定模型下一轮学什么。

虽然 dynamics model 的监督学习形式允许使用 off-policy transition，但“可以训练”不等于“在当前策略访问区域中预测准确”。如果训练数据主要来自 random policy，而 MPC policy 已经进入了新的状态区域，就会产生 distribution shift。

6.4 实验结果

表 2: Problem 3 三个环境的最终 evaluation return

环境	最终 Eval Return
Obstacles	-24.70
Reacher	-269.65
HalfCheetah	426.32

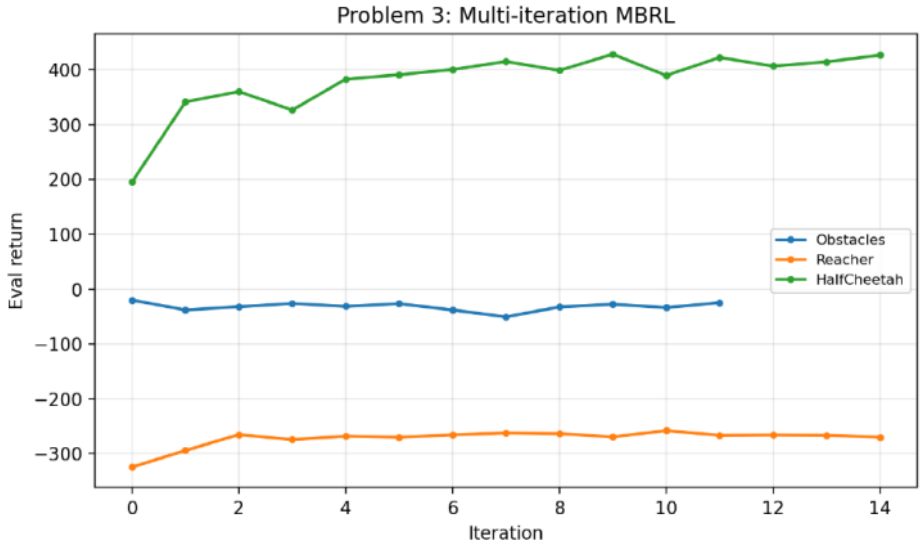


图 2: Problem 3：三个环境上的 multi-iteration MBRL return curves。

6.5 结果分析

三个环境都达到或超过作业预期。Obstacles 最终为 -24.70，Reacher 为 -269.65，HalfCheetah 为 426.32。

HalfCheetah 的提升最明显。初始时模型只看到随机动作产生的数据，控制器对未来的预测能力有限。随着 MPC 策略收集更多 on-policy data，模型逐渐覆盖当前策略真正访问的状态区域，因此 return 持续提高。

Reacher 的结果相对平稳，因为它需要非常精确的末端位置控制。一个小的状态预测误差可能直接影响最终距离 reward。Obstacles 更偏向避障，只要模型能够排除明显危险的动作序列，return 就能达到比较稳定的范围。

这里还要注意，on-policy data collection 并不意味着每轮只保留最新数据。代码把新数据加入 replay buffer，同时保留历史数据，这能减少模型对最近一小批轨迹的过拟合和灾难性遗忘。

“这还是 on-policy 吗？replay buffer 里明明有旧数据。”新数据由当前策略收集，所以数据采集过程具有 on-policy 更新特征；模型训练使用累积 replay buffer，属于混合分布监督学习。这里强调的是持续补充当前策略访问区域，而不是严格只用最新轨迹。

7 Model-Based RL Problem 4：MPC 超参数消融

7.1 题目要求

Problem 4 在 Reacher 环境中研究三个超参数：

- ensemble size；
- candidate action sequences 数量；
- MPC planning horizon。

每个参数分别减小一次、增大一次，并和默认配置比较。总共 7 个 run。每次只能改变一个参数，其他参数保持默认。

7.2 对应 YAML 配置

```yaml
# 默认配置：所有消融实验的共同基线。
exp_name: reacher_ablation
ensemble_size: 3
mpc_num_action_sequences: 1000
mpc_horizon: 10

# ensemble size 减小。
exp_name: reacher_ablation_ens1
ensemble_size: 1
mpc_num_action_sequences: 1000
mpc_horizon: 10

# ensemble size 增大。
exp_name: reacher_ablation_ens5
ensemble_size: 5
mpc_num_action_sequences: 1000
mpc_horizon: 10

# candidate action sequences 减小。
exp_name: reacher_ablation_actionseq100
ensemble_size: 3
mpc_num_action_sequences: 100
mpc_horizon: 10

# candidate action sequences 增大。
exp_name: reacher_ablation_actionseq2000
ensemble_size: 3
mpc_num_action_sequences: 2000
mpc_horizon: 10

# planning horizon 减小。
exp_name: reacher_ablation_horizon5
ensemble_size: 3
mpc_num_action_sequences: 1000
mpc_horizon: 5
```

```
# planning horizon 增大。
exp_name: reacher_ablation_horizon15
ensemble_size: 3
mpc_num_action_sequences: 1000
mpc_horizon: 15
```

消融实验最重要的不是“多跑几个配置”, 而是**控制变量**。例如比较 horizon 时, ensemble size 和 action sequence 数量必须保持一致，否则性能变化无法归因于 horizon。

7.3 完整结果

表 3: Problem 4 Reacher MPC 超参数消融结果

配置	最终 Eval Return
Default: ensemble=3, actionseq=1000, horizon=10	-260.99
ensemble=1	-277.50
ensemble=5	-263.76
actionseq=100	-321.99
actionseq=2000	-257.40
horizon=5	-231.44
horizon=15	-279.04

7.4 Ensemble size：代码参数如何影响算法

在 Problem 2 中，动作序列评分为

$$\widehat{R}_k = \frac{1}{E}\sum_{i=1}^{E} \widehat{R}_{i,k},$$

其中 E 是 ensemble size，$\widehat{R}_{i,k}$ 是第 i 个模型对第 k 条动作序列的累计 reward 预测。

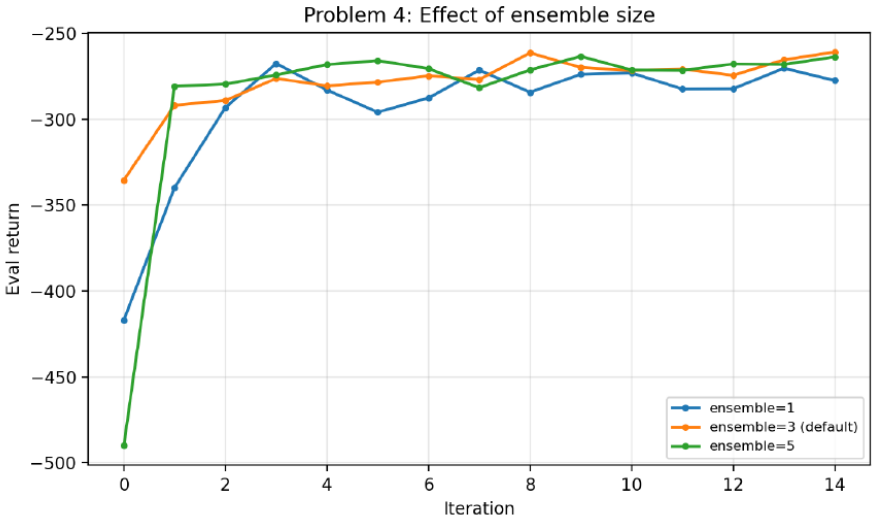


Figure 3: Problem 4: ensemble size 对性能的影响。

图 3: Problem 4：ensemble size 对 Reacher 性能的影响。

ensemble size 从 1 增加到 3 时，最终 return 从 -277.50 提升到 -260.99。这说明多个模型平均可以降低单模型偶然误差的影响。

但从 3 增加到 5 后，结果为 -263.76，没有继续改善。原因可能有三个：第一，模型之间的差异不足，增加两个相似模型提供的信息有限；第二，平均预测只能降低方差，不能消除所有模型共同存在的系统性偏差；第三，更多模型增加计算开销，但不一定改善动作排序。

所以 ensemble size 不是越大越好，而是存在明显的边际收益递减。

7.5 Candidate action sequences：随机搜索覆盖率

Random Shooting 的本质是在动作序列空间中进行 Monte Carlo search。候选序列越多，采到高质量序列的概率通常越高，但每条序列都要进行 $E \times H$ 次模型预测。

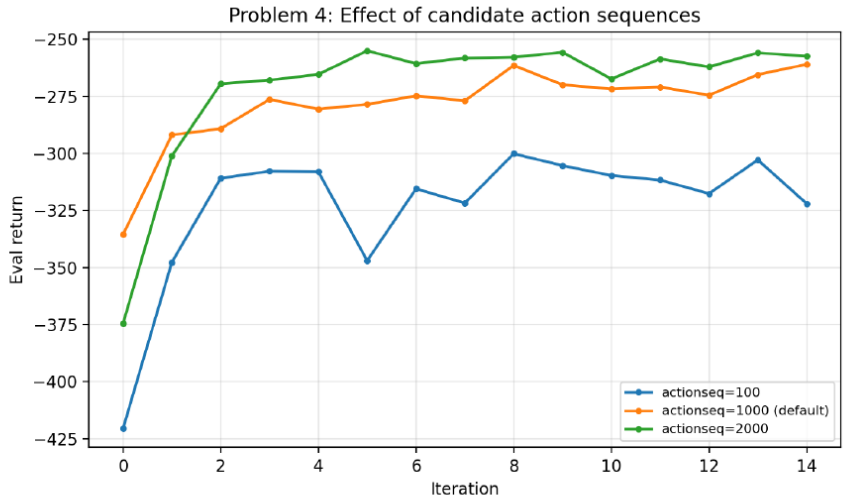


Figure 4: Problem 4: candidate action sequences 数量对性能的影响。

图 4: Problem 4: candidate action sequences 数量对性能的影响。

action sequences 只有 100 条时，最终 return 是 -321.99，明显最差。因为 Random Shooting 完全依赖随机采样，样本太少时很难覆盖到高质量动作区域。

从 100 增加到 1000 后，性能明显提升到 -260.99。进一步增加到 2000 后，只小幅提升到 -257.40。

这说明候选序列数量增加初期很有效，但达到一定规模后会出现边际收益递减。因为最优区域已经有较高概率被覆盖，再增加采样主要带来计算成本，而不是同等幅度的性能收益。

7.6 Planning horizon：看得更远与误差累积的权衡

若模型每一步存在小误差 ϵ_t，多步 rollout 后误差不是简单停留在当前一步，而会影响下一步模型输入。因此实际误差可能呈现复合传播：

$$\widehat{s}_{t+2} = f(\widehat{s}_{t+1}, a_{t+1}), \qquad \widehat{s}_{t+1} \neq s_{t+1}.$$

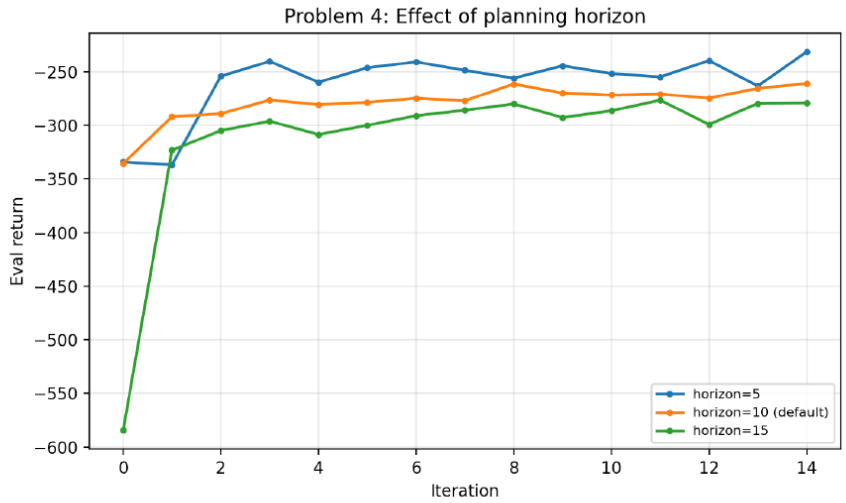


Figure 5: Problem 4: MPC planning horizon 对性能的影响。

图 5: Problem 4: MPC planning horizon 对性能的影响。

horizon 等于 5 时效果最好, 最终 return 为 -231.44; 默认 horizon 等于 10 时为 -260.99; horizon 增加到 15 后反而下降到 -279.04。

理论上，更长 horizon 能考虑更远未来，应该有利于长期决策。但 learned model 不是完全准确的。第一步预测出现偏差以后，第二步就会在错误状态上继续预测，后面的状态和 reward 估计会越来越不可靠。

因此，horizon 的选择体现了两个目标的冲突：短 horizon 预测可靠，但可能过于短视；长 horizon 考虑长远，但容易受到 compounding model error 影响。本实验中 Reacher 对精确状态预测非常敏感，所以较短 horizon 更好。

8 Model-Based RL Problem 5：Cross-Entropy Method

8.1 题目要求

Problem 5 要求实现 CEM action selection，并比较：

- Random Shooting；
- CEM，2 次迭代；
- CEM，4 次迭代。

Random Shooting 只采样一次；CEM 会反复选出 elite sequences，并根据 elite 的均值和标准差更新下一轮采样分布。

8.2 完整核心代码

```
elif self.mpc_strategy == "cem":
    # elite_mean 和 elite_std 表示当前高斯采样分布参数。
    elite_mean, elite_std = None, None

    # CEM 进行多轮迭代，每轮都会重新评估和拟合分布。
    for i in range(self.cem_num_iters):

        # 第 0 轮还没有 elite 分布，直接使用初始随机动作序列。
        if i == 0:
            candidate_action_sequences = action_sequences

        # 后续轮次从上一轮 elite 拟合出的高斯分布中采样。
        else:
            candidate_action_sequences = np.random.normal(
                loc=elite_mean,
                scale=elite_std,
                size=(
                    self.mpc_num_action_sequences,
                    self.mpc_horizon,
                    self.ac_dim,
                ),
            )

            # 将动作裁剪到环境允许范围。
            candidate_action_sequences = np.clip(
                candidate_action_sequences,
                self.env.action_space.low,
                self.env.action_space.high,
            )

        # 用 learned dynamics model 评估所有候选序列。
        rewards = self.evaluate_action_sequences(
            obs,
            candidate_action_sequences,
        )

        # 取 reward 最大的 cem_num_elites 条序列。
        elite_indices = np.argsort(rewards)[
            -self.cem_num_elites:
        ]
        elites = candidate_action_sequences[elite_indices]

        # 用 elite 的逐时间步均值和标准差拟合新分布。
        new_elite_mean = np.mean(elites, axis=0)
        new_elite_std = np.std(elites, axis=0)

        # 第一轮直接初始化分布。
        if elite_mean is None:
```

```
            elite_mean = new_elite_mean
            elite_std = new_elite_std

        # 后续轮次做平滑更新，防止采样分布变化过快。
        else:
            elite_mean = (
                self.cem_alpha * new_elite_mean
                + (1 - self.cem_alpha) * elite_mean
            )
            elite_std = (
                self.cem_alpha * new_elite_std
                + (1 - self.cem_alpha) * elite_std
            )

    # MPC 最后仍然只执行优化后动作序列的第一步。
    return np.clip(
        elite_mean[0],
        self.env.action_space.low,
        self.env.action_space.high,
    )
```

8.3 逐轮算法过程

假设每轮采样 K 条动作序列，选择其中 M 条 elite。第 j 轮可概括为：

1. 从分布 $\mathcal{N}(\mu_j, \sigma_j^2)$ 采样动作序列；
2. 用 learned dynamics model 计算每条序列的累计 reward；
3. 选出 top-M elite sequences；
4. 计算 elite 的均值 μ_{elite} 和标准差 σ_{elite}；
5. 使用平滑系数更新：

$$\mu_{j+1} = \alpha\mu_{\text{elite}} + (1-\alpha)\mu_j,$$

$$\sigma_{j+1} = \alpha\sigma_{\text{elite}} + (1-\alpha)\sigma_j.$$

张量/数组形状检查

`candidate_action_sequences`	(K, H, d_a)；
`rewards`	$(K,)$；
`elites`	(M, H, d_a)；
`elite_mean`	(H, d_a)；
`elite_std`	(H, d_a)。

注意：均值和标准差不是一个标量，而是对每个未来时间步、每个动作维度分别拟合。

8.4 分析

CEM 和 Random Shooting 的共同点是，它们都不需要对 reward 关于 action 求梯度，都是黑盒优化方法。
区别在于 Random Shooting 每次只采样一批，然后直接选最好的一条。它不会利用“哪些区域已经表现较好”这一信息。
CEM 第一轮也从随机动作序列开始，但会保留 top elite sequences。假设优秀序列的第一个动作大多靠近 0.5，那么下一轮的第一个动作就更倾向于从 0.5 附近采样；如果优秀序列在某个动作维度上的变化很小，标准差也会收缩，搜索会更集中。
代码中的 cem alpha 用于平滑更新。如果完全采用最新 elite 的均值和标准差，分布可能过快收缩，早早陷入一个偶然出现的局部区域。平滑更新能保留一部分上一轮分布，避免搜索过度激进。
最后返回 elite mean 的第一个动作，仍然符合 MPC 的 receding horizon 原则。CEM 优化的是完整动作序列，但系统只执行第一步，下一时刻重新运行 CEM。

为什么要 clip？ 高斯分布的采样没有上下界，即使均值在合法范围内，也可能采到超出环境动作空间的值。因此必须用 `np.clip` 裁剪。
为什么 elite std 不能太快变成 0？ 标准差接近 0 时，后续采样几乎没有探索能力，算法可能过早收敛。实际实现中常加入最小标准差或平滑更新。

8.5 实验结果

表 4: Problem 5 HalfCheetah 上 Random Shooting 与 CEM 对比

方法	最终 Eval Return
Random Shooting	426.32
CEM，2 iterations	557.08
CEM，4 iterations	1099.83

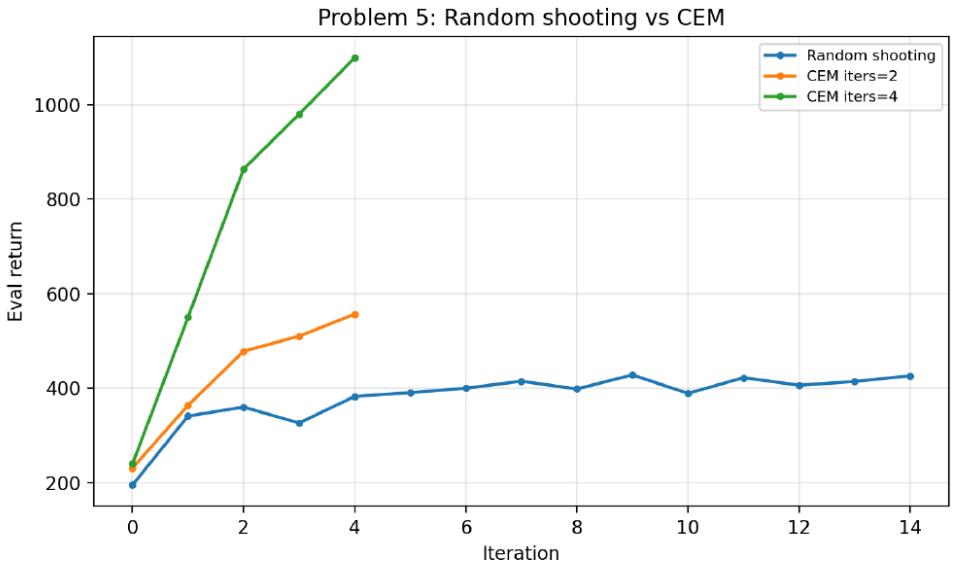


Figure 6: Problem 5: Random shooting 与 CEM 的 return 曲线对比。

图 6: Problem 5：Random Shooting 与 CEM 的 return 曲线。

8.6 结果分析

Random Shooting 最终 return 为 426.32。CEM 迭代 2 次后提升到 557.08；迭代 4 次后进一步达到 1099.83，提升非常明显。

这说明在 HalfCheetah 的连续动作空间中，planner 的搜索策略与 dynamics model 本身同样重要。即使使用同一个 learned model，不同的动作优化方法也会产生巨大性能差距。

CEM 4 次迭代明显优于 2 次，是因为分布有更多机会逐步集中到高 reward 区域。但更多迭代也意味着每一个真实环境时间步都要执行更多轮模型 rollout，计算成本同步增加。

所以在真实机器人控制中，不能只看最终 return，还要考虑控制周期。例如系统要求每 20 毫秒输出一个动作时，CEM 迭代次数必须受实时预算限制。

9 Model-Based RL Problem 6：Dyna-Style 与 MBPO

9.1 题目要求

Problem 6 将 learned dynamics model 与 HW3 的 SAC 结合。模型不再直接通过 MPC 输出最终动作，而是生成额外 synthetic transitions，加入 SAC replay buffer。

需要比较三组：

- model-free SAC：model rollout length $= 0$；
- Dyna-style：model rollout length $= 1$；
- MBPO：model rollout length $= 10$。

9.2 代码一：使用 learned model 生成 synthetic rollout

```
def collect_mbpo_rollout(
    env,
    mb_agent,
    sac_agent,
    ob,
    rollout_len=1,
):
    # 创建列表，保存模型生成的 synthetic transitions。
    obs, acs, rewards, next_obs, dones = [], [], [], [], []

    # 生成 rollout_len 步模型 rollout。
    for _ in range(rollout_len):

        # SAC actor 根据当前 observation 选择动作。
        ac = sac_agent.get_action(ob)

        # ensemble 中每个 dynamics model 都预测一次 next observation。
        ensemble_next_obs = np.stack(
            [
                mb_agent.get_dynamics_predictions(
                    i,
                    ob[None],
                    ac[None],
                )[0]
                for i in range(mb_agent.ensemble_size)
            ],
            axis=0,
        )

        # 对 ensemble 的 next observation 预测结果取平均。
        next_ob = np.mean(
            ensemble_next_obs,
            axis=0,
        )

        # 使用环境 reward function 计算 synthetic reward。
        rew, _ = env.get_reward(
            next_ob[None],
            ac[None],
        )
        rew = rew[0]

        # 保存当前 synthetic transition。
        obs.append(ob)
        acs.append(ac)
        rewards.append(rew)
        next_obs.append(next_ob)

```

```
        # 当前简化实现中不预测终止，因此 done 固定为 False。
        dones.append(False)

        # 下一步从模型预测出的 next_ob 继续。
        ob = next_ob

    # 返回可插入 replay buffer 的 batch。
    return {
        "observation": np.array(obs),
        "action": np.array(acs),
        "reward": np.array(rewards),
        "next_observation": np.array(next_obs),
        "done": np.array(dones),
    }
```

9.3 代码二：将 synthetic transitions 加入 SAC replay buffer

```
# rollout length > 0 时，才使用 learned model 生成 synthetic data。
if sac_config["mbpo_rollout_length"] > 0:

    # 从真实 replay buffer 中随机采一个真实 observation 作为起点。
    start_ob = replay_buffer.sample(1)[
        "observations"
    ][0]

    # 使用 learned dynamics model 生成 synthetic rollout。
    rollout = collect_mbpo_rollout(
        env,
        mb_agent,
        sac_agent,
        start_ob,
        sac_config["mbpo_rollout_length"],
    )

    # 将 synthetic transitions 插入 SAC replay buffer。
    sac_replay_buffer.batched_insert(
        observations=rollout["observation"],
        actions=rollout["action"],
        rewards=rollout["reward"],
        next_observations=rollout["next_observation"],
        dones=rollout["done"],
    )

# 从 SAC replay buffer 采样真实与 synthetic 数据混合后的 batch。
batch = sac_replay_buffer.sample(
    sac_config["batch_size"]
)

# HW3 的 SAC update 接收 torch Tensor，因此进行类型转换。
sac_agent.update(
    ptu.from_numpy(batch["observations"]),
    ptu.from_numpy(batch["actions"]),
    ptu.from_numpy(batch["rewards"]),
    ptu.from_numpy(batch["next_observations"]),
    ptu.from_numpy(batch["dones"]),
```

```
    i,
)
```

9.4 数据流与形状

若 rollout length 为 L：

张量/数组形状检查

`start_ob`	$(d_s,)$，从真实 replay buffer 采样；
`ac`	$(d_a,)$，由 SAC actor 给出；
`ensemble_next_obs`	(E, d_s)；
`next_ob`	$(d_s,)$，ensemble 平均；
返回 `observation`	(L, d_s)；
返回 `action`	(L, d_a)；
返回 `reward`	$(L,)$；
返回 `next_observation`	(L, d_s)；
返回 `done`	$(L,)$。

9.5 分析

MBPO 和 MPC 都使用 learned dynamics model，但它们使用模型的方式不同。
MPC 是在当前时刻直接使用模型比较动作序列，然后输出一个动作。MBPO 不直接让模型决定最终动作，而是让模型生成额外 transition，再由 SAC 从这些数据中学习 actor 和 critic。
collect mbpo rollout 首先从真实 replay buffer 选一个起点。这个设计很重要，因为如果从任意虚构状态开始，模型可能立刻进入完全没有数据支持的区域。以真实状态为起点，可以让 rollout 初期更可靠。
之后 SAC actor 根据当前状态选择动作, ensemble models 分别预测下一状态, 代码对预测结果取平均, 再用已知 reward function 计算 reward。于是得到一条 synthetic transition：

$$(s_t, a_t, \widehat{r}_t, \widehat{s}_{t+1}).$$

如果 rollout length 大于 1，下一步会从 $\hat{s}_{t+1}$ 继续。这意味着第一步在真实状态上预测，第二步在模型状态上预测，第三步在更深的模型状态上预测。rollout 越长，误差累积越明显。

生成的数据被插入 SAC replay buffer。之后 SAC 不区分某条样本来自真实环境还是 learned model，而是统一用于更新 Q function 和 policy。因此，如果 synthetic data 足够准确，就能以很低的真实交互成本提供大量训练信号；如果模型数据偏差太大，也会污染 critic target。

从 0 到 1 步 rollout 的意义尤其重要：一步模型数据没有递归使用预测状态，因此误差通常较小，却已经能够显著扩充训练样本。这也是 Dyna-style 方法常常能带来明显收益的原因。

为什么 done 固定为 False? 这是当前作业中的简化实现。更完整的方法应该训练终止模型，或者依据环境规则判断 synthetic terminal。若真实任务有重要终止条件而始终设为 False，可能产生不真实的长轨迹。

为什么不直接随机选择一个 ensemble model，而是取平均? 取平均可以降低预测方差，但可能产生“多个可行状态的平均值”，这个平均状态未必物理可行。更复杂的 MBPO 实现常对每条 rollout 随机选择一个模型，或使用 uncertainty 筛选。

9.6 实验结果

表 5: Problem 6 HalfCheetah 上 SAC、Dyna-style 与 MBPO 对比

方法	Model rollout length	最终 Eval Return
Model-free SAC	0	438.72
Dyna-style	1	816.41
MBPO	10	902.84

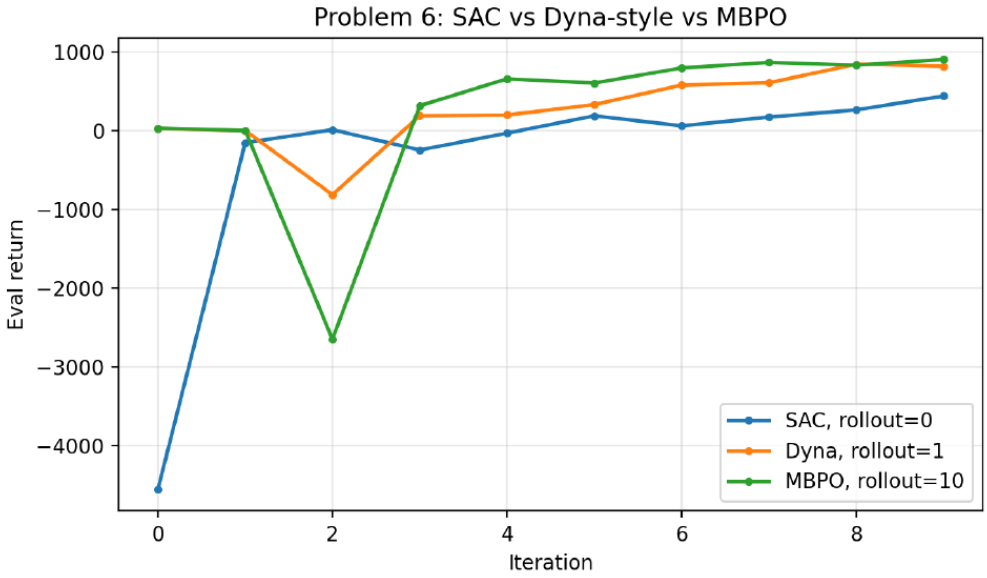


图 7: Problem 6：不同 model rollout length 对 SAC 训练效果的影响。

9.7 结果分析

纯 model-free SAC 的最终 return 为 438.72。加入一步 model rollout 后，Dyna-style 提升到 816.41。使用 10 步 rollout 的 MBPO 进一步达到 902.84。

从 0 步到 1 步的提升最大，说明少量高质量 synthetic transitions 已经能显著提高样本利用率。一步 rollout 没有递归累积模型误差，因此数据通常比较可靠。

从 1 步增加到 10 步后仍然提升，但增幅小于从 0 到 1。这说明更长 rollout 一方面生成更多训练数据，另一方面开始承受模型误差累积，两种效应互相抵消。

本实验中 length=10 的结果最好，说明当前 HalfCheetah dynamics model 在这一长度范围内生成的 synthetic data 仍然总体有益。但不能据此推断 rollout 越长越好。如果继续增加到 20、50 或更长，模型轨迹可能严重偏离真实环境，最终反而降低 SAC 性能。

10 把六道题连完整

10.1 从代码角度看完整数据流

1. 真实环境产生 (s_t, a_t, s_{t+1})；
2. Problem 1 用它训练 $f_\theta(s_t, a_t) \approx s_{t+1} - s_t$；
3. Problem 2 使用 f_θ 对候选动作序列进行 rollout；

4. Problem 3 使用当前 MPC policy 收集新数据，减少 distribution shift；
5. Problem 4 调整模型数、搜索宽度和规划深度；
6. Problem 5 用 CEM 将“盲目随机搜索”改成“利用 elite 信息的迭代搜索”；
7. Problem 6 把模型从 planner 变成 data generator，辅助 SAC。

10.2 所有关键结果汇总

表 6: HW4 主要实验结果汇总

题目	实验或设置	最终结果
Problem 1	Default dynamics model：1 layer，hidden size 32	loss 0.1623
Problem 1	Wider dynamics model：1 layer，hidden size 64	loss 0.1177
Problem 1	Deeper dynamics model：2 layers，hidden size 32	loss 0.1542
Problem 2	Obstacles，one-iteration Random Shooting MPC	-30.26
Problem 3	Obstacles，iterative MBRL	-24.70
Problem 3	Reacher，iterative MBRL	-269.65
Problem 3	HalfCheetah，iterative MBRL	426.32
Problem 4	Reacher，actionseq=100	-321.99
Problem 4	Reacher，horizon=5	-231.44
Problem 5	HalfCheetah，CEM 4 iterations	1099.83
Problem 6	HalfCheetah，MBPO rollout length=10	902.84

11 个人思考与总结

11.1 Model-Based RL 的真正优势

完成这次作业以后，我认为 Model-Based RL 的核心优势不是“模型能预测未来”这句话本身，而是它可以把一部分昂贵的真实试错替换成便宜的模型试错。

在 MPC 中，智能体可以在执行动作之前，在模型中比较大量候选方案；在 MBPO 中，模型可以快速生成额外 transition，让 SAC 得到更多更新机会。这对于机器人、自动驾驶和工业控制尤其重要，因为真实环境交互可能昂贵、缓慢，甚至有安全风险。

11.2 贯穿全部实验的风险：模型误差累积

Problem 4 中 horizon=15 比 horizon=5 更差；Problem 6 中 rollout length 从 1 增加到 10 后仍有收益，但收益明显变小。这两组结果都说明，模型预测越长，误差越容易逐步累积。

模型误差最危险的地方，不只是单步预测有偏差，而是策略可能主动利用模型漏洞，选择在模型中看起来回报很高、在真实环境中却很差的动作。这通常称为 model exploitation。

11.3 为什么 uncertainty-aware 方法值得继续做

ensemble 不仅可以取平均，也可以通过模型之间的 disagreement 估计不确定性：

$$U(s,a) = \mathrm{Var}_{i=1,\dots,E}\left[f_{\theta_i}(s,a)\right].$$

若不同模型预测差异很大，说明该区域数据不足。后续可以：

- 在 MPC objective 中惩罚高不确定性动作序列；
- 根据不确定性自适应缩短 rollout length；
- 优先在高 disagreement 区域采集真实数据；
- 对 synthetic transitions 进行置信度加权。

11.4 最终收尾

最后总结一下，这次 HW4 并不是简单地多训练一个神经网络，而是完整展示了模型学习、动作规划、数据收集、搜索优化和 model-free policy learning 如何相互配合。
Problem 1 解决“模型怎样学”；Problem 2 解决“模型怎样用于规划”；Problem 3 解决“模型怎样跟随策略分布更新”；Problem 4 研究“模型和 planner 应该使用多大规模”；Problem 5 解决“怎样更高效地搜索动作”；Problem 6 解决“怎样用模型提高 SAC 的样本效率”。
MPC 通过每一步重新规划减轻长期误差；CEM 通过 elite selection 提高搜索效率；MBPO 通过有限长度的 synthetic rollout 在数据量和模型可靠性之间取得平衡。
因此，我对 Model-Based RL 最核心的理解是：它的目标不是让模型预测得无限远，而是要在模型利用程度和模型可信度之间找到合适的平衡。

A 可能的问题

1. **为什么预测 delta 而不是 next state?**
   因为连续控制中相邻状态变化通常更平滑，delta 的分布更集中，学习难度更低。

2. **为什么输入和 target 都要 normalization?**
   为了减弱不同维度数量级差异，改善梯度尺度和优化稳定性。

3. **ensemble 为什么能提高稳健性?**
   多个独立模型的平均可以降低单模型预测方差，同时 disagreement 可用于估计不确定性。

4. **MPC 为什么只执行第一步?**
   执行后可以获得真实状态并重新规划，从而及时修正模型预测偏差。

5. **为什么 action sequences 越多不一定越好?**
   搜索覆盖率提高后会出现边际收益递减，而计算量近似随候选序列数线性增长。

6. **为什么 horizon 越长反而可能更差?**
   因为后续预测建立在先前预测状态上，模型误差会复合传播。

7. **CEM 为什么比 Random Shooting 更有效?**
   CEM 利用 elite sequences 更新采样分布，不会每轮都从整个空间盲目搜索。

8. **Dyna-style 和 MBPO 的区别是什么?**
   Dyna-style 在本实验中对应 1-step model rollout；MBPO 使用更长但仍受控的模型 rollout 生成更多 synthetic data。

9. **synthetic data 会不会污染 SAC?**
   会。如果模型误差过大或 rollout 太长，critic 会学习到错误 target，因此需要控制 rollout length 和 synthetic data 比例。

10. **下一步最值得改进什么?**
    使用 ensemble disagreement 做 uncertainty-aware planning、adaptive rollout length 和 synthetic data filtering。